# NAVBOT-EN01

# BLE

# communication protocol

Table of Contents

## Revision History

| DOCUMENT HISTORY | | | |
|---|---|---|---|
| **SOFTWARE REVISION** | **CHANGES MADE** | **REVISION DATE** | **EDITED BY** |
| **1.0** | **Add BLE protocol** | **5/1/2025** | **XiaoleChen** |
| **1.1** | **Modify the document structure,and add WIFI connection command** | **5/19/2025** | **XiaoleChen** |
| **1.2** | **Correct the errors in version 1.1** | **5/23/2025** | **XiaoleChen** |
| **1.3** | **Add JSON and define the protocol for reading status.** | **7/3/2025** | **XioaleChen** |
| **1.4** | **Add and modify JSON key-value pairs** | **7/17/2025** | **XioaleChen** |
| | | | |
| | | | |
| | | | |

# 1. BLE information

BLE NAME ：
navbot_en01-******

Service UUID:
6E400011-B5A3-F393-E0A9-E50E24DCCA9E

Send data characteristic UUID:
6E400012-B5A3-F393-E0A9-E50E24DCCA9E

Receive data characteristic UUID:
6E400013-B5A3-F393-E0A9-E50E24DCCA9E

# 2. Data frame agreement

| Byte | Name | Description |
|---|---|---|
| **Byte 1** | Header1 | **Fixed 0x55** |
| **Byte 2** | Header2 | **Fixed 0xAA** |
| **Byte 3** | **Command** | **The control command of this data frame, reference 3.Command** |
| **Byte 4** | **Remaining** | **The number of frames remaining after synthesizing the complete data** |
| **Byte 5** | **Null** | **Retain** |
| **Byte 6 ~ Byte 20** | Data | |

# 3. Command

The third byte is the command byte, which determines the function of the data from the 6th to the 20th byte. The current definition is as follows:

| Command | Name | Explanation |
|---|---|---|
| 0x00 | CMD_MANEUVER | Robot maneuver, at this time the meaning of the data frame reference 3.0.CMD_MANEUVER. |
| 0x01 | CMD_WIFI | Connect the robot to the wireless network. The data frame refers to 3.1.CMD_WIFI. |
| 0x02 | CMD_JSON | JSON transmission, byte 6 to byte 20 represent the JSON body. Refer to 3.2. CMD_JSON |
| | | |
| | | |

# 3.0. CMD_MANEUVER

When byte3(command) is equal to 0x00, it indicates that bytes 6 to 20 control the maneuverability of the robot. At this time, the data should follow the following rules:

| **Byte** | **Name** | **Description** |
|---|---|---|
| **Byte 1** | Header1 | **Fixed 0x55** |
| **Byte 2** | Header2 | **Fixed 0xAA** |
| **Byte 3** | **Command** | **0x00** |
| **Byte 4** | **Remaining** | **0x00** |
| **Byte 5** | **Null** | **Retain** |
| **Byte 6** | roll | **Rolling Angle, value range -30 to 30.** |
| **Byte 7** | height | **The height of the robot, with a value range of 32 to 85.** |
| **Byte 8** | linear_H | **Linear speed, with a value range of -200 to 200.** |
| **Byte 9** | linear_L | |
| **Byte 10** | angular | **The angular velocity of the robot, with a value range of -100 to 100.** |
| **Byte 11** | stable | **The stable state of the robot: 1 on, 0 off** |
| **Byte 12** | mode | **Fixed 0x01** |
| **Byte 13** | dir | **The movement direction of the robot is defaulted to 0, jump:1** |
| **Byte 14** | joy_y | **The Y-axis data of the joystick, with a value range of -100 to 100.** |
| **Byte 15** | joy_x | **The X-axis data of the joystick, with a value range of -100 to 100.** |
| **Byte 16 ~ Byte 20** | **Null** | **Idle** |

### 3.0.1. Data frame demonstration：

55 AA 00 00 00 00 26 0000 00 01 00 00 00 00 00 00 00 00 00 // height = 0x26(38), stable = 1
After sending this instruction, the robot starts and the leg height is 38mm.

55 AA 00 00 00 00 30 0000 00 01 00 00 00 00 00 00 00 00 00 // height = 0x30(48), stable = 1 ,After sending this instruction, the robot starts and the leg height is 48mm.

55 AA 00 00 00 00 30 0000 00 00 00 00 00 00 00 00 00 00 00 // height = 0x26(38), stable = 0 ,After sending this instruction, the robot stops and the leg height is 48mm.

55 AA 00 00 00 00 26 0000 00 01 00 00 00 05 00 00 00 00 00 // height = 0x26(38), stable = 1 ,joy_y = 0x05(5) ,
After sending this instruction, the robot moves forward at a speed of 5.

55 AA 00 00 00 00 26 0000 00 01 00 00 00 FA 00 00 00 00 00 // height = 0x26(38), stable = 1 ,joy_y = 0xFA(-5),
After sending this instruction, the robot moves backward at a speed of 5.

55 AA 00 00 00 00 26 0000 00 01 00 00 00 00 05 00 00 00 00 // height = 0x26(38), stable = 1 ,joy_x = 0x05(5) ,
After sending this instruction, the robot rotates clockwise with an angular velocity of 5.

55 AA 00 00 00 00 26 0000 00 01 00 00 00 00 FA 00 00 00 00 // height = 0x26(38), stable = 1 ,joy_x = 0xFA(-5),
After sending this instruction, the robot rotates counterclockwise with an angular velocity of 5.

55 AA 00 00 00 00 26 0000 00 01 00 00 01 00 00 00 00 00 00 // height = 0x26(38), stable = 1 ,dir = 0x01,
sending this instruction, the robot jumps and is ready.
55 AA 00 00 00 00 26 0000 00 01 00 00 00 00 00 00 00 00 00 // height = 0x26(38), stable = 1 ,dir = 0x00,
After sending this instruction, the robot jumps in place.

## 3.1. CMD_WIFI

When byte3(command) is equal to 0x01, The next 6 to 20 bytes are the information of the wifi, which includes the SSID (wifi name) and password of the wifi. The wifi name and password should be in the ASCLL character set. Then, the structure of the data should follow the following rules:

| Byte | Name | Description |
|---|---|---|
| **Byte 1** | Header1 | **Fixed 0x55** |
| **Byte 2** | Header2 | **Fixed 0xAA** |
| **Byte 3** | **Command** | **0x01** |
| **Byte 4** | **Remaining** | ***** |
| **Byte 5** | **Null** | **Idle** |
| **Byte 6 ~ Byte 20** | **data** | **This piece of data stores the name and password of the wifi, with the name first and the password second, separated by a TAB('\t',ASCLL=9) character in between. 3.1.1.Data frame demonstration The data demonstration format is listed** |

### 3.1.1. Data frame demonstration：

55 AA 01 00 00 6E 61 76 62 6F 74 09 49 4A 4B 4C 4D 4E 4F 50

The example data above is a complete BLE data frame; Starting from byte 6 to byte 11, they are all valid ASCLL characters. However, byte 12 (already marked) is 0x09, which represents a TAB character in ASCLL encoding. Therefore, before this byte (0x09), it should be the wifi name, and after this byte (0x09), it is the password. Then we can come up with the wifi name as "navbot" and the password as "12345678".

## 3.2. CMD_JSON:

This command data frame format is:

| Byte | Name | Description |
|---|---|---|
| **Byte 1** | Header1 | **Fixed 0x55** |
| **Byte 2** | Header2 | **Fixed 0xAA** |
| **Byte 3** | **Command** | **0x02** |
| **Byte 4** | **Remaining** | ***** |
| **Byte 5** | **Null** | **Idle** |
| **Byte 6 ~ Byte 20** | **Body data** | **JSON body,Transmission can be subcontracted.** |

### 3.2.1. JSON protocol :

以键名 ”type” 作为不同的命令区分，当前定义的 type 键值有以下：

| Type value：(string) | 说明 |
|---|---|
| “set_wifi” | Set up Wi-Fi information, refer to 3.2.1.1 |
| “sys_web_socket_server” | Set up the cloud server, referring to 3.2.1.2 |
| “sys_restart” | Restart the robot, refer to 3.2.1.3 |
| “get_device_info” | Obtain robot information, refer to 3.2.1.4 |
| “off_servo” | Shut off the steering gear, refer to 3.2.1.5 |
| “on_servo” | Start the servo motor, refer to 3.2.1.6 |
| “calibrate_servo” | Calibrate the servo motor based on the current status, referring to 3.2.1.7 |

#### 3.2.1.1. sys_wifi

```
{
    “type”:”sys_wifi”;
    “ssid”:”**********”;
    “password”:”**********”;
    “state”: “server”; // or “client”; or ”close”;
}
```

#### 3.2.1.2. sys_web_socket_server

```
{
```

```
    “type”:”sys_web_socket_server”;
    “host”:”**********”;
    “port”:4488; //int
    “url”: “*****”;

}
```

#### 3.2.1.3. sys_restart

```
{
    “type”:”sys_restart”;
}
```

#### 3.2.1.4. get_device_info

```
{
    “type”:”get_device_info”;
}
```

#### 3.2.1.5. off_servo

```
{
    “type”:”off_servo”;
}
```

#### 3.2.1.6. on_servo

```
{
    “type”:”on_servo”;
}
```

#### 3.2.1.7. calibrate_servo

```
{
    “type”:”calibrate_servo”;
}
```

# 4. Subcontracting

Since the BLE Bluetooth data frame is only 20 bytes, excluding the first 5 bytes that have been occupied, the valid data provided to the user is only 15 bytes. However, in many scenarios,

the data is far greater than 15 bytes. For instance, the characters of the wifi name and password are often larger than 15 bytes. At such times, we can use the 4th byte (the Remaining part) for Subcontracting operations. It indicates how many frames of data that follow need to be merged with this frame of data, as shown in the following example:

Suppose the wifi name is “American_network_ABCDEFG” and the password is “0123456789”, Then the following data frames should be sent:

55 AA 01 02 00 41 6D 65 72 69 63 61 6E 5F 6E 65 74 77 6F 72
55 AA 01 01 00 6B 5F 41 42 43 44 45 46 47 09 30 31 32 33 34
55 AA 01 00 00 35 36 37 38 39 00 00 00 00 00 00 00 00 00 00